



Instituto Tecnológico de Costa Rica
IC-2001 Estructuras de Datos

II Proyecto Programado
Sistema de Modelado de Galaxias
Grafos y Archivos

Elaborado por :Benjamin Ortiz Quispe

Profesora : Raquel Mora Rojas

Primer semestre de 2026

Descripción del problema

Los Guardianes de la Galaxia son un equipo de héroes especializado en resolver conflictos intergalácticos y disputas internas entre los distintos planetas que conforman el cosmos. Su labor los obliga a desplazarse constantemente de un sistema estelar a otro, de modo que el transporte entre puntos distantes no es un detalle logístico menor, sino una parte central de su vida cotidiana: muchas veces, de la rapidez con que lleguen a un conflicto depende su resolución.

El problema surge de la complejidad inherente del universo en el que operan. El cosmos está formado por incontables planetas, estrellas y nebulosas interconectados mediante una red de rutas espaciales que se entrelazan sin un orden evidente. Algunos tramos son directos y predecibles, mientras que otros forman caminos enredados, con atajos como agujeros de gusano y nodos que enlazan con múltiples destinos a la vez. Navegar esa red de forma manual resulta impráctico y propenso a elegir trayectos costosos o lentos cuando existen alternativas mejores.

Ante esta situación, el capitán de la nave ha ordenado el desarrollo de un sistema computacional que modele la red de transporte intergaláctico y permita tomar decisiones de viaje informadas. El sistema debe representar el conjunto de planetas y rutas como un grafo (donde cada planeta es un nodo y cada ruta una conexión con su costo y tiempo asociados), determinar las conexiones más eficientes entre puntos y mantener un registro ordenado tanto de las rutas existentes como de los viajes realizados por la flota.

Para construirlo, la información base del modelo se obtiene de un servicio web que entrega los datos en formato JSON, los cuales deben consumirse mediante la librería curl. A partir de esos datos, el sistema debe representar el grafo usando listas o matrices de adyacencia; implementar una modificación del algoritmo de Kruskal —una versión que parte de una lista de arcos ordenada de forma aleatoria— para hallar una red galáctica de bajo costo que mantenga todos los sistemas conectados; registrar en estructuras auxiliares las rutas visitadas por las naves; y guardar los resultados y configuraciones en archivos de salida con formato de texto. El producto final es una herramienta que convierte la inmensidad caótica del cosmos en una red navegable y optimizada.

Justificación

El problema planteado por el capitán surge de una situación bastante clara, los Guardianes de la Galaxia necesitan desplazarse constantemente entre distintos sistemas, y hacerlo manualmente dentro de una red tan grande de planetas y rutas termina siendo poco

práctico. Elegir un camino incorrecto puede implicar más tiempo de viaje o mayores costos, por lo que el sistema desarrollado busca precisamente facilitar esa toma de decisiones.

Para representar la red galáctica se decidió utilizar un grafo, donde cada galaxia corresponde a un nodo y cada ruta a una arista. Cada conexión almacena tanto su costo como el tiempo de viaje, permitiendo modelar la información que el problema requiere. Además, el grafo se considera no dirigido, ya que las rutas espaciales pueden recorrer en ambos sentidos, lo que representa de manera más natural el comportamiento de la red.

La representación interna se implementó mediante listas de adyacencia. Esta decisión resulta conveniente debido a que la red presentada es dispersa: existen galaxias con muchas conexiones y otras con muy pocas. Una matriz de adyacencia habría desperdiciado memoria almacenando gran cantidad de conexiones inexistentes, mientras que la lista únicamente conserva aquellas rutas que realmente existen. Adicionalmente, la lista se reconstruye automáticamente cuando el grafo cambia, garantizando que cualquier operación se realice sobre información actualizada.

Uno de los objetivos principales del sistema es determinar rutas eficientes entre dos galaxias. Para ello se implementó el algoritmo de Dijkstra, el cual permite obtener el camino de menor costo dentro de un grafo con pesos positivos. La elección de este algoritmo responde directamente a la necesidad del problema, ya que garantiza encontrar la ruta óptima según el costo de viaje.

Sin embargo, el sistema no se limita a calcular una única ruta. Se genera un recorrido sobre la red completa y otro sobre la red optimizada obtenida mediante Kruskal, permitiendo comparar ambas soluciones. De esta forma el usuario no solo obtiene una respuesta, sino también información que le ayuda a tomar decisiones. La visualización mediante raylib complementa este proceso al mostrar gráficamente las rutas calculadas, facilitando la comprensión del resultado.

El requisito más importante del proyecto consiste en implementar una variante del algoritmo de Kruskal utilizando una lista de aristas en orden aleatorio. El sistema respeta completamente esta condición, ya que las aristas se procesan en el mismo orden en que son recibidas desde la API, sin realizar un ordenamiento previo por costo.

Para evitar ciclos se implementó una estructura Union-Find con compresión de caminos y unión por rango, lo que permite determinar eficientemente si una arista puede agregarse al árbol. De esta manera se construye una red conexa y libre de ciclos utilizando exactamente $N-1$ aristas. La decisión de mantener tanto el grafo original como el árbol generado por Kruskal permite evaluar la efectividad de la optimización. El sistema compara ambas redes y genera reportes que evidencian las diferencias encontradas, permitiendo demostrar el beneficio obtenido.

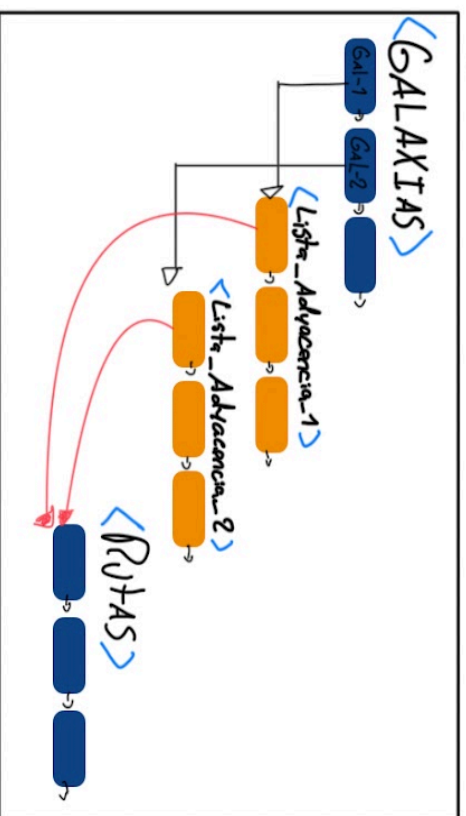
La obtención de datos se realiza mediante libcurl, consumiendo los servicios web especificados en el enunciado. Los datos descargados se procesan para reconstruir el modelo de la galaxia cada vez que se inicia el programa.

Se decidió utilizar un parser JSON propio basado en expresiones regulares con el objetivo de mantener el proyecto autocontenido y sencillo de compilar. Esto evita dependencias adicionales y permite que todo el sistema pueda construirse mediante un único make, aspecto que resulta especialmente útil en un entorno académico.

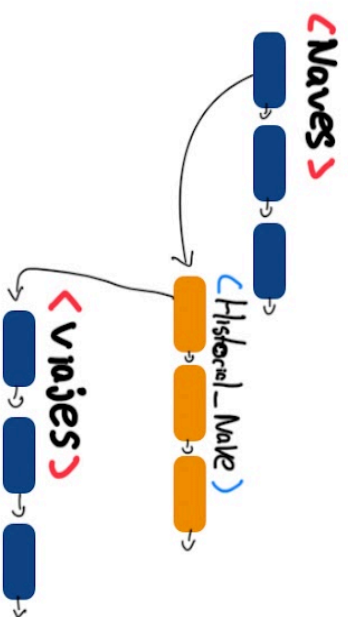
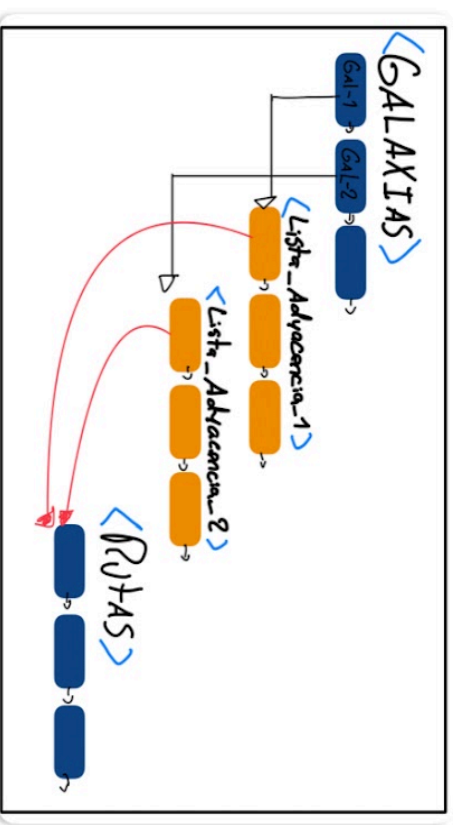
Para el registro de información se implementó la estructura Viaje, permitiendo que cada nave mantenga su propio historial de recorridos. Asimismo, se almacenan las rutas generadas tanto en la red original como en la optimizada, facilitando posteriores análisis y comparaciones.

Finalmente, el sistema genera distintos reportes en archivos de texto. El esquema de nombres evita sobreescrituras y permite conservar los resultados de cada ejecución. Los reportes estadísticos resumen el comportamiento de la red y muestran el impacto de las optimizaciones realizadas, proporcionando evidencia del funcionamiento del sistema.

GRAFO_PRINCIPAL



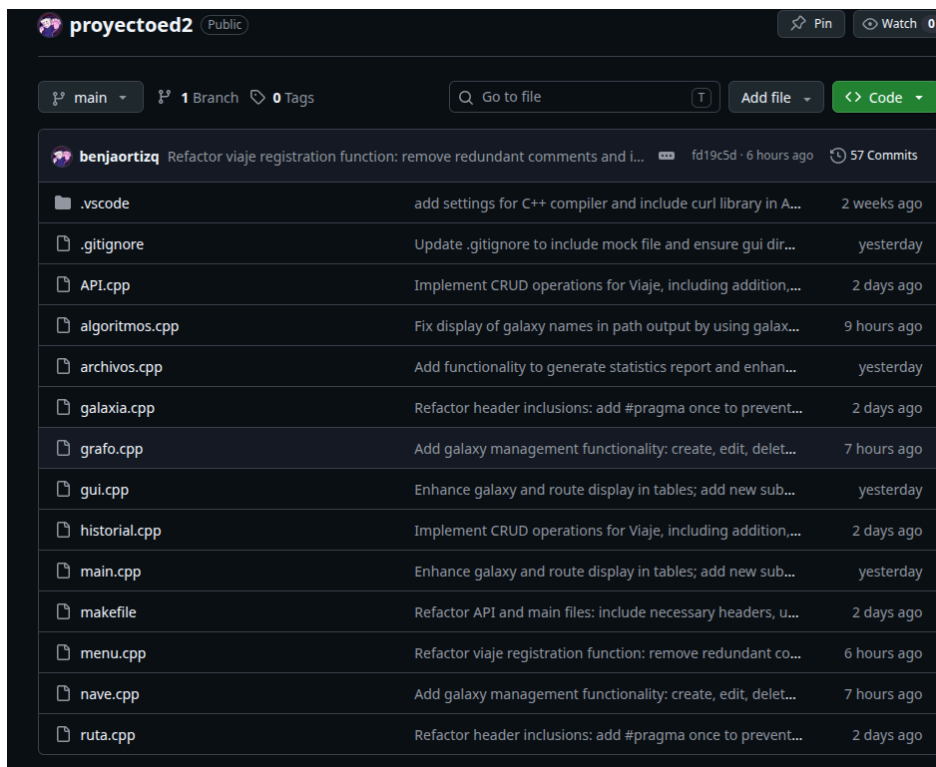
GRAFO_K



Solución al problema :

Se decidió, primero, usar la plataforma de GitHub como herramienta de versionado de software, ya que permitía llevar un control claro de cada avance del proyecto y de los cambios que se iban haciendo sobre el código. Esto fue útil no sólo para registrar el progreso de manera ordenada, sino también para volver a versiones anteriores en caso de error, comparar modificaciones y mantener un historial de trabajo más transparente. Además, GitHub facilitó bastante el trabajo remoto, ya que permitía subir los cambios desde cualquier lugar, sincronizar el proyecto entre distintas máquinas y colaborar sin depender de estar físicamente en el mismo espacio. En un proyecto de este tamaño, esa organización sí marca una diferencia, porque ayuda a trabajar con más seguridad y con menos riesgo de perder avances.

[LINK DEL PROYECTO](https://github.com/benjaortizq/proyectoed2) (<https://github.com/benjaortizq/proyectoed2>)



Se decidió hacer el programa en C++17 y modelar el universo como un grafo no dirigido y ponderado. Las galaxias son los nodos y las rutas son las aristas, cada una con costo y tiempo de viaje. Sobre esa base se pueden hacer consultas de caminos mínimos, construir una red optimizada con Kruskal, registrar historial de viajes, mostrar la información en pantalla, dibujarla con raylib y exportar reportes a archivos de texto. O sea, no solo se guarda información: se usa para tomar decisiones.

La estructura general del proyecto parte desde las entidades básicas. Cada una cumple un rol muy definido. Galaxia representa los nodos de la red; Ruta, las conexiones entre galaxias; Nave, la flota que hace los recorridos; y Viaje, el historial de lo que cada nave ha hecho. Las cuatro siguen una misma lógica de diseño: constructor vacío, constructor con parámetros, método de edición y método de impresión. Eso ayuda bastante porque todo se maneja igual desde los menús, los reportes y los algoritmos. Además, el id de cada entidad se trata como clave fija, porque cambiarlo rompería las búsquedas y las relaciones entre estructuras. Esa decisión evita errores y mantiene el modelo consistente.

La Galaxia guarda sus datos principales: código, nombre, tipo, descripción y coordenadas espaciales. Las coordenadas no están ahí de adorno; son las que después usa la parte gráfica para ubicar cada nodo en la ventana. La Ruta conecta dos galaxias y guarda el costo, el tiempo y si está activa. Algo importante es que el grafo no está dirigido, así que una ruta puede recorrer en ambos sentidos. Eso hace que el modelo sea más natural para el problema, porque no obliga a duplicar rutas que realmente representan la misma conexión. La Nave incluye capacidad, velocidad, estado y un historial propio de viajes. Ese historial no está puesto como algo decorativo: permite consultar rápido lo que hizo cada nave sin tener que filtrar la lista global de viajes cada vez. Y Viaje funciona como registro real de actividad, guardando la nave, la ruta, la fecha, el costo real y si el viaje salió bien, además de algunos nombres ya resueltos para mostrar todo más fácil.

Encima de esas estructuras básicas se construye el grafo. La parte central del sistema es Grafo, que mantiene los nodos, las rutas y la lista de adyacencia. La lista de adyacencia no se eligió por capricho: esta red es dispersa, incompleta, y una matriz habría gastado memoria innecesaria guardando conexiones que no existen. Con una lista, solo se almacena lo que realmente hay, y eso vuelve más eficiente tanto la búsqueda como los algoritmos que recorren la red. Cada vez que el grafo cambia, la lista se reconstruye para que siempre refleje el estado actual. Eso es clave, porque así las consultas nunca trabajan con información vieja.

El grafo también concentra parte importante de la lógica de búsqueda y de modificación. Permite encontrar galaxias por id o por código, buscar rutas, imprimir adyacencias, calcular conectividad y ordenar galaxias según cuántas rutas tienen. Pero además trae sus propias operaciones CRUD: agregar, editar y eliminar galaxias y rutas. Aquí hay una idea que sí vale la pena remarcar: la integridad referencial se cuida en todo momento. No se permite agregar una ruta si sus extremos no existen. Tampoco se deja una galaxia borrada con rutas colgando. Cuando se elimina una galaxia, se borran también todas las rutas que la tocan. Eso evita inconsistencias y mantiene la red limpia.

Para la optimización de la red se usa Kruskal con Union-Find. Esa parte es importante porque no es solo “hacer un árbol”, sino construir una versión más barata de la red manteniendo todo conectado. La estructura Union-Find usa compresión de caminos y unión por rango, así que

detectar si una arista forma ciclo es rápido. Esto hace que Kruskal funcione bien incluso si la red crece. El sistema mantiene dos grafos en memoria: Principal, que es la red completa, y k, que es la red optimizada. Tener ambos en paralelo no es redundante; es lo que permite comparar la red original con la optimizada, calcular rutas en ambas y demostrar si la optimización realmente aporta algo.

Los algoritmos principales del proyecto son Kruskal y Dijkstra. Kruskal construye el árbol de expansión mínima, y Dijkstra encuentra rutas de menor costo entre dos galaxias. Dijkstra se aplica sobre el grafo completo y sobre el grafo optimizado, así que el usuario puede ver la diferencia entre moverse por toda la red o por una versión reducida. Eso hace que la solución sea más completa, porque no se queda solo en “calcular un camino”, sino que compara dos escenarios y muestra cuál conviene más. La ruta mínima no se calcula una sola vez para luego olvidarla; se guarda en vectores de rutas generadas, uno para el grafo principal y otro para el de Kruskal. Eso después se usa en reportes y en la comparación ruta por ruta.

La capa de archivos sirve para dejar evidencia de lo que se calculó en la sesión. No es una base de datos persistente, sino una salida ordenada de reportes. Se generan tres tipos de archivos: rutas mínimas, árbol de expansión y estadísticas. Cada archivo tiene nombre con fecha y consecutivo, así no se sobrescribe nada. Eso es una decisión práctica porque permite conservar varias corridas del programa sin perder información. Lo bueno de esto es que los mismos métodos que imprimen en pantalla se reutilizan para escribir a disco, así no se duplica código y se mantiene consistencia entre lo que el usuario ve y lo que se guarda.

La parte de API también tiene mucho peso dentro de la solución. El programa descarga los datos desde un servicio web y reconstruye el modelo cada vez que arranca. Para eso usa libcurl, que se encarga de traer el JSON desde los endpoints remotos. En lugar de meter una librería adicional para JSON, se hizo un parser propio más simple y autocontenido. Esa decisión puede verse pequeña, pero en un proyecto académico sí ayuda bastante: reduce dependencias, hace que el compilado sea más limpio y permite trabajar todo con una sola unidad de traducción. Además, el arranque incluye reintentos si la conexión falla, lo que lo vuelve más robusto.

La carga de información se hace por etapas. Primero se obtiene el grafo base, luego se completan los datos de las galaxias, después se cargan naves y viajes, y al final se asigna el historial a cada nave. Ese orden no es casualidad: cada paso depende del anterior. Una vez todo está cargado, se reconstruye la adyacencia del grafo principal y se genera el árbol optimizado con Kruskal. Así, al terminar el arranque, el sistema ya tiene todo listo para consultas, reportes y CRUD.

La interfaz gráfica con raylib complementa el resto del proyecto. No se hizo solo por estética. Sirve para mostrar la red de forma clara y resaltar los caminos calculados por Dijkstra. También permite dibujar el árbol de Kruskal de forma jerárquica. Eso ayuda mucho a entender visualmente lo que está pasando con los datos, especialmente cuando se comparan la red

completa y la optimizada. En vez de ver solo listas y costos, el usuario ve la estructura de la red y los recorridos destacados.

El sistema de menús es el que conecta todo con el usuario. Desde ahí se accede a consultas, reportes y operaciones CRUD. Las consultas permiten ver rutas desde una galaxia, calcular el camino más barato entre dos puntos, generar el árbol de conexiones y revisar el historial de viajes de una nave. Los reportes exportan la información calculada. Y el menú del grafo permite crear, editar, ver y eliminar galaxias, rutas y naves. Cada operación válida entrada, evita datos inválidos y mantiene el estado del sistema coherente. Cuando se modifica una galaxia o una ruta, el grafo se actualiza y Kruskal se recalca, porque si no, la red optimizada quedaría desfasada.

Uso del programa

El proyecto se compila con make, que genera el ejecutable proyecto (requiere g++ con soporte para C++17 y las librerías libcurl y raylib instaladas.. Para correrlo basta con ejecutar ./proyecto desde la terminal. Es indispensable contar con conexión a internet, ya que al arrancar el programa descarga todo el modelo de galaxias, rutas, naves e historial desde la API remota (si no existiese un archivo grafo.txt anteriormente creado). Si no hay red, el sistema reintenta la conexión durante 60 segundos y, de no lograrlo, se cierra avisando del problema. También conviene que exista la carpeta files/ en el directorio del programa, pues es ahí donde se guardan los reportes de texto generados. Al iniciar correctamente, se muestra una pantalla de bienvenida con animaciones de carga ("Conectando con EndPoint...", "Obteniendo datos de Galaxias...", etc.) y luego aparece el menú principal con cuatro opciones: Consultas, Reportes, Grafo y Salir. Toda la navegación se hace escribiendo números y presionando Enter, y en casi todos los submenús el 0 sirve para volver atrás o cancelar. Por último, hay que tener presente que los cambios hechos con el CRUD viven solo en memoria durante la sesión: no se envían de vuelta a la API, así que conviene generar los reportes antes de salir si se quiere conservar evidencia del trabajo realizado.

Funcionalidades

1. Consultas

- Rutas desde una galaxia: muestra todas las conexiones (rutas) que salen de una galaxia elegida, es decir, su lista de adyacencia.
- Ruta de menor costo entre dos galaxias: aplica el algoritmo de Dijkstra para encontrar el camino más barato entre un origen y un destino. Lo calcula dos veces —sobre la red completa y sobre la red optimizada— para comparar ambos trayectos, e incluye una visualización gráfica (raylib) con la ruta resaltada en rojo.

- Árbol de conexiones: calcula el árbol de expansión mínima con el algoritmo de Kruskal y lo muestra en una visualización jerárquica tipo "árbol genealógico".
- Historial de viajes por nave: lista las naves y muestra todos los viajes realizados por la que se elija.

2. Reportes

Genera tres archivos de texto en la carpeta files/, numerados automáticamente por fecha para no sobrescribir:

- Rutas más cortas generadas: vuelca todos los caminos calculados durante la sesión con sus costos y tiempos.
- Árbol de expansión: exporta el árbol de Kruskal con todas sus galaxias y rutas.
- Estadísticas: reporte comparativo que cuenta las rutas calculadas en cada red, lista las galaxias más conectadas y emite un veredicto ruta por ruta sobre cuál red es más eficiente (la original o la optimizada).

3. Administración del Grafo (CRUD)

Permite crear, ver, editar y eliminar las tres entidades del modelo:

- Galaxias (nodos), Rutas (aristas) y Naves (flota).
- Incluye una opción especial para registrar viajes, que entrelaza nave + galaxia origen + ruta para construir un nuevo registro de historial.
- El sistema mantiene la integridad referencial (al borrar una galaxia se eliminan sus rutas; no se admiten rutas hacia galaxias inexistentes) y recalcula automáticamente la red optimizada tras cada cambio.

Minuta de Trabajo

El trabajo fue individual , asi que estos tiempos solo son de Benjamin Ortiz

Componente	Descripción	Tiempo (Horas)
Entidades base (galaxia.cpp, ruta.cpp, nave.cpp, viaje)	Diseño e implementación de las estructuras principales del sistema. Se desarrollaron los constructores, métodos de edición, impresión y manejo de identificadores únicos, además de establecer las relaciones básicas entre las entidades del modelo.	7
Grafo (grafo.cpp)	Desarrollo de la estructura principal del sistema, incluyendo la lista de adyacencia, funciones de búsqueda por identificador y código, operaciones CRUD e implementación de reglas de integridad referencial para mantener la consistencia del modelo.	13
Algoritmos (algoritmos.cpp)	Implementación y pruebas de los algoritmos de Kruskal y Dijkstra. Se desarrolló la estructura Union-Find con compresión de caminos y unión por rango, así como los métodos de cálculo y reconstrucción de caminos mínimos.	15
API (API.cpp)	Implementación de la comunicación con la API mediante libcurl, construcción del parser JSON propio, procesamiento de la información descargada y desarrollo del sistema de carga por etapas y reintentos ante fallos de conexión.	13
Archivos (archivos.cpp)	Desarrollo del sistema de generación de reportes, manejo de archivos de salida, creación de nombres automáticos con fecha y consecutivos, y exportación de rutas, árboles de expansión y estadísticas del sistema.	5
GUI (gui.cpp)	Implementación de la visualización gráfica utilizando raylib. Se desarrolló el dibujo de la red galáctica, el resaltado de rutas calculadas por Dijkstra y la representación jerárquica del árbol generado por Kruskal.	13

Menús (menu . cpp)	Diseño de la navegación del programa, validación de entradas del usuario, implementación de las opciones CRUD y actualización automática del modelo después de cada modificación realizada por el usuario.	7
Integración (main . cpp, makefile)	Integración de todos los módulos del proyecto, configuración del proceso de compilación, realización de pruebas generales, corrección de errores y verificación del funcionamiento completo del sistema.	10
Documentación	Elaboración del resumen técnico, justificación de las decisiones tomadas, diagramas de estructuras y documentación general del funcionamiento del proyecto.	8

Total de horas : 91



Análisis de Resultados

El sistema desarrollado sí cumple con el objetivo principal del enunciado: tomar la red galáctica y convertirla en un grafo que se pueda consultar, analizar y optimizar de forma útil. La idea no era solo representar datos, sino permitir que los Guardianes pudieran tomar decisiones de viaje con base en rutas reales, costos y tiempos, en vez de andar resolviendo todo a mano.

En cuanto al modelado del cosmos, se implementó exactamente como pedía el problema: las galaxias se manejan como nodos y las rutas como aristas con costo y tiempo asociados. Para eso se usaron Galaxia y Ruta, y el grafo se construyó como no dirigido y ponderado, lo cual tiene sentido porque una ruta espacial puede recorrer en ambos sentidos. Además, la red se guarda con listas de adyacencia, que era una forma bastante natural de representar un sistema disperso como este, y esa lista se reconstruye cada vez que hay cambios para que el modelo no quede desactualizado.

La parte más importante del proyecto, a nivel algorítmico, fue Kruskal. Se implementó una versión modificada que trabaja con una lista de aristas en el orden en que llegan, sin asumir que ya vienen ordenadas por costo. Con eso se obtiene una red secundaria que sigue conectada, sin ciclos y con exactamente $N-1$ aristas. Para controlar ciclos se usó Union-Find con compresión de caminos y unión por rango, que hace este proceso bastante eficiente. El resultado es un árbol de expansión válido que funciona como versión optimizada de la red original. Tener Principal y k en memoria al mismo tiempo fue una decisión útil, porque permite comparar ambas redes y no depender de una sola estructura.

Para encontrar rutas eficientes entre dos galaxias se usó Dijkstra con cola de prioridad. Eso permitió calcular el camino de menor costo de forma consistente, tanto en la red completa como en la red optimizada. Esa comparación era clave, porque el proyecto no solo debía devolver una ruta, sino mostrar si la red optimizada realmente ofrece ventaja frente a la original. Por eso también se guardan los caminos generados durante la sesión en estructuras auxiliares, de manera que luego se puedan comparar y reportar.

El manejo de rutas y viajes también quedó resuelto de forma ordenada. Se implementó CRUD para galaxias, rutas y naves, y además se manejó el historial de viajes con la estructura Viaje. Cada nave conserva su propio historial, lo que hace más fácil consultar sus recorridos sin tener que filtrar toda la lista global cada vez. También se mantuvo la integridad referencial: si se borra una galaxia, se eliminan sus rutas asociadas, y no se permiten rutas hacia nodos que no existen. Eso evita inconsistencias en el grafo.

El sistema genera archivos de texto con los resultados importantes: rutas mínimas, árbol de expansión y estadísticas. Los reportes se guardan en files/ con fecha y consecutivo, para no sobrescribir información anterior. Eso deja evidencia de cada sesión y permite revisar lo que se calculó sin depender de la ejecución actual.

Conclusiones y Recomendaciones

El desarrollo de este proyecto permitió construir un sistema funcional y completo para modelar una red de transporte intergaláctico, demostrando que las estructuras de datos y los algoritmos de grafos son herramientas concretas para resolver problemas reales de optimización y consulta.

La implementación de Dijkstra y Kruskal sobre dos grafos en paralelo no solo cumplió con los requisitos del enunciado, sino que aportó un nivel de análisis comparativo que le da valor real al sistema: el usuario no se limita a obtener una ruta, sino que puede evaluar si la red optimizada representa una mejora respecto a la original. A esto se suma la agregue la visualización gráfica con raylib, elementos que en conjunto elevan el proyecto de un ejercicio teórico y lo convierten en una herramienta navegable y con evidencia de sus resultados.

Desde el punto de vista de diseño, las decisiones tomadas a lo largo del proyecto resultaron coherentes entre sí. El uso de listas de adyacencia fue la representación más adecuada para una red dispersa, la integridad referencial del grafo evitó inconsistencias durante las operaciones CRUD, y el esquema de reportes con fecha y consecutivo garantizó que cada ejecución dejará un registro preservado. El proyecto logra lo que se propuso desde el principio: convertir la complejidad de una red galáctica en un sistema estructurado, consultable y optimizado, donde cada módulo tiene una responsabilidad clara y todos trabajan juntos para sostener el modelo sin redundancias ni ambigüedades.

Bibliografía

GeeksforGeeks. (2026, Enero). *Dijkstra's algorithm*.
<https://www.geeksforgeeks.org/dsa/dijkstras-shortest-path-algorithm-greedy-algo-7/>

GeeksforGeeks. (2025, Diciembre). *Kruskal's minimum spanning tree (MST) algorithm*.
<https://www.geeksforgeeks.org/dsa/kruskals-minimum-spanning-tree-algorithm-greedy-algo-2/>

raylib. (n.d.). *Raylib examples*. <https://www.raylib.com/examples.html>

Making a game in C++ to prove I'm a real programmer. (n.d.). *YouTube*.
<https://www.youtube.com/watch?v=wkVqe0l6cWA>